Satisfiability / SAT Problem

→ Given a proposition F (Boolean Formula), if there exists an assignment to unit propositions such that F evaluates to true, F is satisfiable.

→ If no such assignment exist, F is unsatisfiable (UNSAT)

$$F \rightarrow (p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \wedge \neg p)$$

F is satisfiable?

→ $F' \Rightarrow (p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p)$
$\wedge (p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r)$

NP Complete

$$\left.\begin{array}{c} p \equiv F \\ \downarrow \\ q \equiv F \\ \downarrow \\ r \equiv F \end{array}\right\} \quad \left.\begin{array}{c} p \equiv T \\ \downarrow \\ q \equiv T \\ \downarrow \\ r \equiv T \end{array}\right\} \alpha$$

Given a problem instance (Boolean formula) I, we are interested to find a solution S (an assignment to unit production of I):

Algorithm $C(I, S) \Rightarrow$ True

CNF : (Conjunctive Normal Form)

$$A = \bigwedge_i \bigvee_j L_{i,j}$$

$L_{i,j}$ → is a literal

$$A = \underbrace{(p \vee \neg q)}_{\text{Clause}} \wedge \underbrace{(\neg p \vee r)}_{\text{Clause}}$$

$L_{i,j} \rightarrow p / \neg p$

atom negation of atom

A clause is $L_1 \vee L_2 \vee \cdots \vee L_n$ where $n \geq 0$,

$n=0 \Rightarrow$ empty clause $\quad L_i \in \{L_i, \neg L_i\}$

$$(L_{11} \vee L_{12} \vee L_{13}) \wedge (L_{21} \vee L_{22} \vee L_{23})$$

K-SAT Problem

$$\underbrace{(L_1 \vee L_2 \vee \cdots \vee L_K)}_{\text{Clause}} \quad \text{K-clause}$$

Finding SAT of a Boolean Formula where each clause is of size $\leq K$

$\underbrace{(p \vee \neg q)}_{\text{2-clause}} \quad \underbrace{(p \vee q \vee r)}_{\text{3-clause}}$

$$\underset{\text{2-clause}}{(p \vee \neg q)} \longrightarrow \underset{\text{3-clause}}{(p \vee p \vee \neg q)}$$

$F = \underbrace{L_1 \vee L_2} \vee \underbrace{L_3 \vee L_4} \Rightarrow$

$(\neg p \vee L_1 \vee L_2) \wedge (p \vee L_3 \vee L_4)$

$(p \rightarrow (L_1 \vee L_2)) \wedge (\neg p \rightarrow (L_3 \vee L_4))$

$L_3 \vee L_4 \vee L_5$

$(\neg q \vee L_3) \quad (q \vee L_4 \vee L_5)$

K-SAT $\quad K \geq 3$

- NP-complete
- P $\quad$ --- $K=2$

Dirn / Approach towards solution to SAT prob

- Unit propagation

$$\underline{L \vee C}$$

If there is a literal $L \in F$, remove $L \vee C$ from F.

Replace $(\neg L \vee C)$ it by C in F.

$$F = \neg\Big(\overset{X}{(p \to q)} \wedge \overset{Y}{(p \wedge q \to r)} \longrightarrow \overset{Z}{(p \to r)}\Big)$$

$$\cancel{= \neg((\neg p \vee q) \wedge (}$$

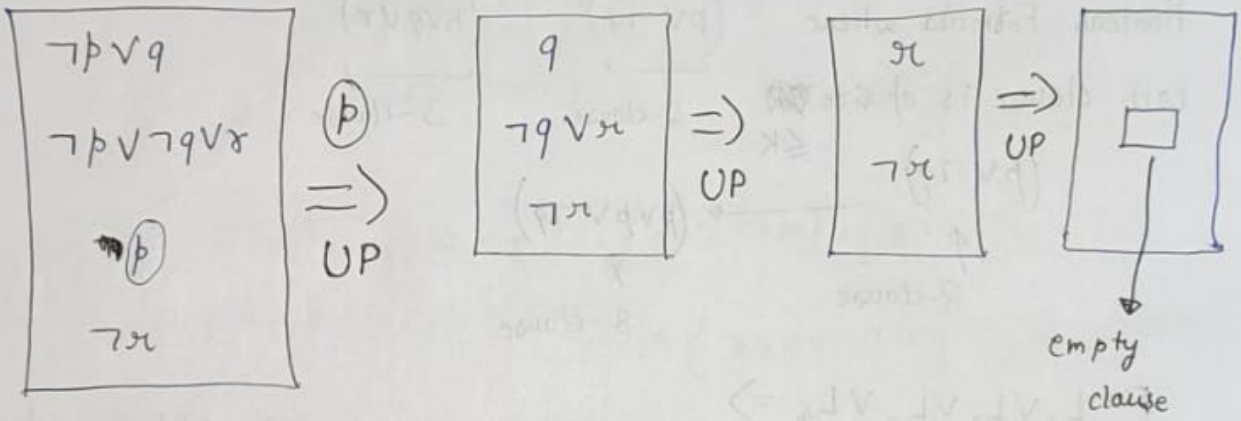
$$F = \neg((X \wedge Y) \to Z)$$

$$= \neg(\neg(X \wedge Y) \vee Z)$$

$$= (X \wedge Y) \wedge \neg Z$$

$$F = \underbrace{(p \to q)}_{C_1} \wedge \underbrace{(p \wedge q \to r)}_{C_2} \wedge \underbrace{\neg(p \to r)}_{p \wedge \neg r}$$

$$(\neg p \vee q) \quad (\neg p \vee \neg q) \vee r \quad \underbrace{p}_{C_3} \quad \underbrace{\neg r}_{C_4}$$



DPLL - Algorithm

Input : F (Boolean Formula)

⇒ set of clauses

Output: Satisfiable / Unsatisfiable

Procedure DPLL(F)

Begin :

□ → empty clause

F = Unit_Propagation (F)

If F is empty (~~F~~ doesn't contains no clause) return SAT

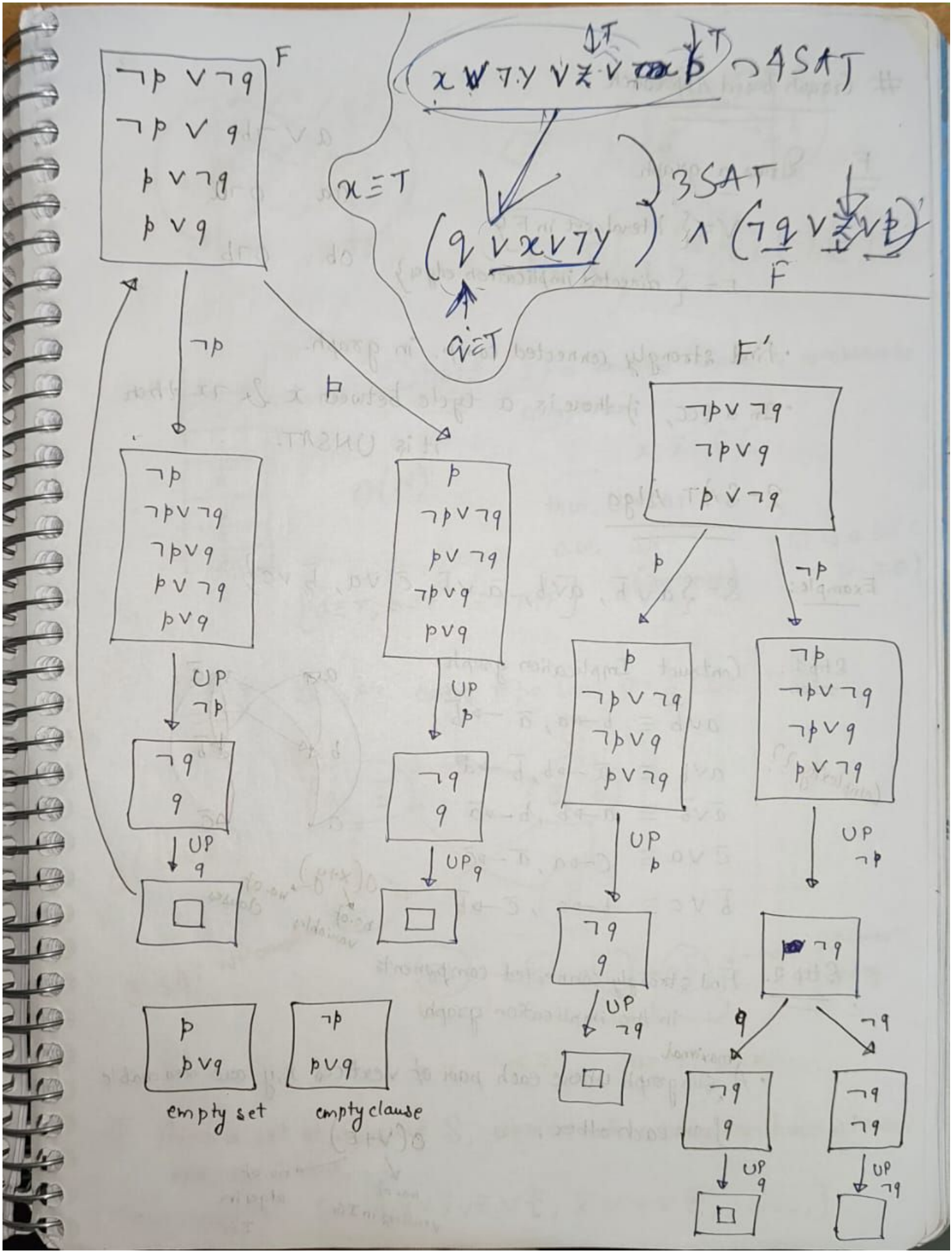
If F contains □ → empty-clause return UNSAT

L := select_literal (F)

If DPLL (F ∪ L) = satisfiable then return SAT

else return DPLL (F ∪ $\bar{L}$)

end:


4SAT
3SAT
x=T
q=T
F
F'
UP
empty set
empty clause

Graph based Approach

$a \vee \neg b$

F Draw a graph

$V = \{$ literal set in F $\}$

$E = \{$ directed implication edges $\}$

a $\quad$ $\neg a$

b $\quad$ $\neg b$

- Find strongly connected comp. in graph.
- In a SCC, if there is a cycle between x & $\neg x$ then it is UNSAT.

2-SAT Algo

Example: $S = \{a \vee \bar{b},\ a \vee b,\ \bar{a} \vee \bar{b},\ \bar{c} \vee a,\ \bar{b} \vee c\}$

Step 1. Construct Implication graph

$a \vee \bar{b} \equiv b \to a,\ \bar{a} \to \bar{b}$

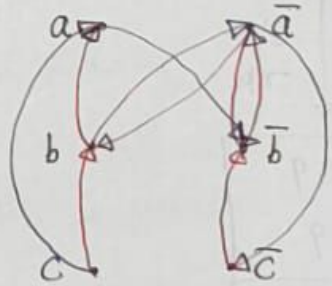
$a \vee b \equiv \bar{a} \to b,\ \bar{b} \to a$

$\bar{a} \vee \bar{b} \equiv a \to \bar{b},\ b \to \bar{a}$

$\bar{c} \vee a \equiv c \to a,\ \bar{a} \to \bar{c}$

$\bar{b} \vee c \equiv b \to c,\ \bar{c} \to \bar{b}$

Complexity ??



$O(x+y)$ → no. of clauses; x → no. of variables

Step 2. Find strongly connected components in the implication graph

- A maximal subgraph where each pair of vertices x, y are reachable from each other.

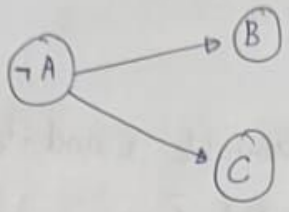
$O(V+E)$

V → no. of vertices in IG; E → no. of edges in IG

$A \vee B \vee C$

$\neg(\neg A) \vee B \vee C$

$\neg A \rightarrow (B \vee C)$



$A \vee B$

$\neg A \rightarrow B$

$C_1 = \neg p_1 \vee p_2$

$C_2 = \neg p_1 \vee p_3 \vee p_5$

$C_3 = \neg p_2 \vee p_4$

$C_4 = \neg p_3 \vee \neg p_4$

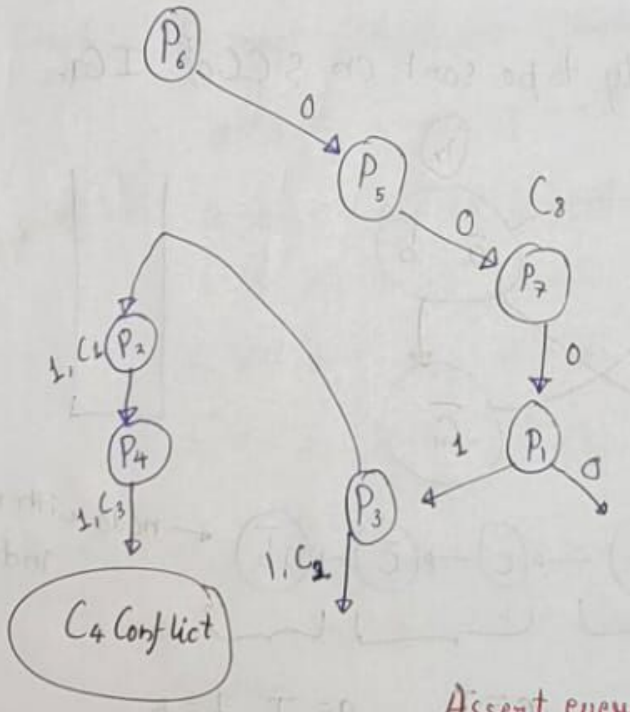
$C_5 = p_1 \vee p_6 \vee \neg p_2$

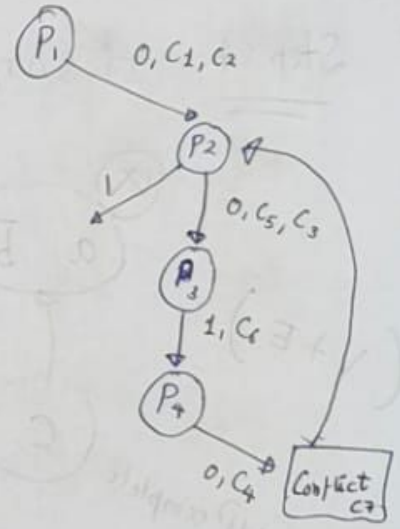
$C_6 = p_2 \vee p_8$

$C_7 = p_2 \vee \neg p_3 \vee p_7$

$C_8 = p_6 \vee \neg p_5$

Q.// Check SAT & sol^n?





Sudoku

Assert every row contains every number

$$\bigwedge_{i=1}^{9} \bigwedge_{n=1}^{9} \bigvee_{j=1}^{9} p(i,j,n)$$

Assert every column contains every number

$$\bigwedge_{j=1}^{9} \bigwedge_{n=1}^{9} \bigvee_{i=1}^{9} p(i,j,n)$$

Sudoku Solver

Number to be filled $\equiv [1, 9]$

(1) row $\equiv i$, column $\equiv j$, digit $\equiv k$ } $i, j, k \in \{1, \ldots, 9\}$

boolean variable $X_{i,j,k}$

DIMACS variable ID $\Rightarrow$ $ID(i,j,k) = 81(i-1) + 9(j-1) + k$

Why? $\Rightarrow$ Ex: $(X_a \vee \neg X_b \vee X_c) \rightarrow \underbrace{1 \ -2 \ 3 \ 0}$

SAT solvers understand CNF in DIMACS format only

So, we have 729 variables

So, $X_{i,j,k}$ is represented as $ID(i,j,k)$.

(2) Cell constraints

for each r, c:

1) Atleast 1 no./dig. $X_{rc1} \vee X_{rc2} \vee \ldots \vee X_{rc9}$

2) Atmost 1 no./dig. $\neg X_{rc1} \vee \neg X_{rc2}$

$\neg X_{rc1} \vee \neg X_{rc3} \ldots \neg X_{rc2} \vee \neg X_{rc3} \ldots$

$\vdots$

$\neg X_{rc1} \vee \neg X_{rc9} \ldots \neg X_{rc2} \vee \neg X_{rc9} \ldots$

Row constraints

for each r, d:

1) Atleast once $X_{r1d} \vee X_{r2d} \vee \ldots \vee X_{r9d}$

2) Atmost once $\neg X_{r1d} \vee \neg X_{r2d}$

$\neg X_{r1d} \vee \neg X_{r3d} \ldots \neg X_{r2d} \vee \neg X_{r3d} \ldots$

$\vdots$

$\neg X_{r1d} \vee \neg X_{r9d} \ldots \neg X_{r2d} \vee \neg X_{r9d} \ldots$

Column Constraints

for each c,d

1) at least once $X_{1cd} \vee X_{2cd} \vee \cdots \vee X_{9cd}$

2) at most once $\neg X_{1cd} \vee \neg X_{2cd}$

$\neg X_{1cd} \vee \neg X_{3cd} \cdots \neg X_{2cd} \vee \neg X_{3cd} \cdots$

$\vdots \qquad \vdots \qquad \vdots \qquad \vdots$

$\neg X_{1cd} \vee \neg X_{9cd} \cdots \neg X_{2cd} \vee \neg X_{9cd} \cdots$

Block Constraints

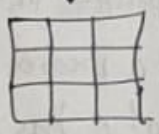
$br \in [0,3)$, $bc \in [0,3)$ } $3 \times 3 = 9$ blocks

for each block (i.e. each br & each bc) & for each d

1) atleast once $X_{(3br+1)(3bc+1)d} \vee X_{(3br+1)(3bc+2)d} \vee X_{(3br+1)(3bc+3)d}$

2) atleast once $\vee X_{(3br+2)(3bc+1)d} \vee X_{(3br+2)(3bc+2)d} \vee X_{(3br+2)(3bc+3)d}$

$\vee X_{(3br+3)(3bc+1)d} \vee X_{(3br+3)(3bc+2)d} \vee X_{(3br+3)(3bc+3)d}$



$b = \begin{bmatrix} X_{(3br+1)(3bc+1)d}, & X_{(3br+1)(3bc+2)d} \cdots \\ \vdots & \vdots \end{bmatrix}$ (array)

for i in b

for j in b

$\neg b[i] \vee \neg b[j]$

Given

for each r,c $\{ X_{r,c,d} \quad (d \,!= 0) \Rightarrow d = given[r-1][c-1]$

* Given a set $S = \{1, 2, ----, n\} = [n] \subseteq \mathbb{N}$

How many simple graphs can be formed using S as the vertex set of the graphs?

$$\Rightarrow 2^{{}^nC_2} \quad {}^nC_2$$

How many of the simple graphs are tree?

We will establish a bijection f

$f : \{$no. of trees with vertex set $S\} \longrightarrow \{$no. of lists of length $n-2$ from $S \rightarrow S^{n-2}\}$

$n = 6$

Each tree from this set will be encoded using an unique set from S^{n-2}

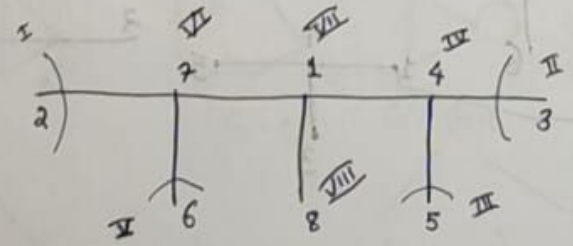
Prüfer Code

A tree T with vertex set, $S = [n] = \{1, 2, ----, n\}$

input :

o/p : an encoding of the tree $f(T) = \{a_1, a_2, ----, a_{n-2}\}$

Procedure : Select least remaining leaf node of T and record its neighbour



$S = [8]$

It 1: least rem leaf $2 \rightarrow 7$, delete 2

It 2: least rem leaf $3 \rightarrow 4$, delete 3

It 3: " " " 5 $\longrightarrow$ 4 , delete 5

It 4: " " " 4 $\longrightarrow$ 1 , delete 4

It 5: " " " 6 $\longrightarrow$ 7 , delete 6

It 6: " " " 7 $\longrightarrow$ 1 , delete 7

[7 4 4 1 7 1] [2 3 5 6 8]

i/p : a prufer code , $f = \{a_1, a_2, ----, a_n\}$

o/p : a tree T corresponding to T

<u>Procedure:</u> 1. Select a_1 from f

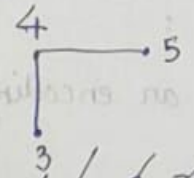
2. Find least remaining no. from S not present in rest of f say x_1

3. connect $a_1 x_1$

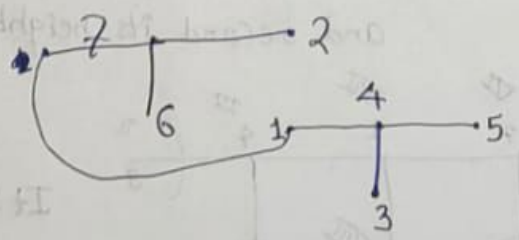
1 2 3 4 5 6 7 8

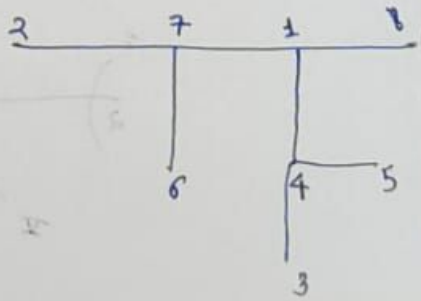
~~7~~ ~~4~~ ~~4~~ 1 7 1 ~~2~~ ~~3~~ ~~5~~ 6 7 8

7 $\longrightarrow$ 2



~~7~~ ~~4~~ ~~4~~ ~~1~~ ~~7~~ ~~1~~ 1 ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~ ~~7~~ 8





Complete Graph

K_n

$u \longleftrightarrow v$

Spanning tree of a graph G is a connected acyclic subgraph that covers all the vertex in G.

$\frac{25-5}{2} = 10$ $\quad 5-1 = 4$

$\frac{n^2-n}{2}$

No. of spanning trees in K_n?

$^nC_2 \; C_{n-1}$ $\propto$

Count is same as no. of trees with vertex set $[n]$

$$\boxed{n^{n-2}}$$

= No. of trees with vertex set $[n]$

= No. of lists of length $n-2$ from set $[n]$

No. of spanning trees in graph G

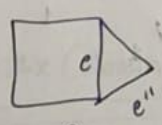
$$T(G) = T(G \setminus e) + T(G \cdot e)$$

contraction

$$e \in E(G)$$

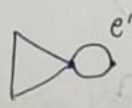
{ merge vertices, remove joining edge }

Example



G

$G \setminus e$



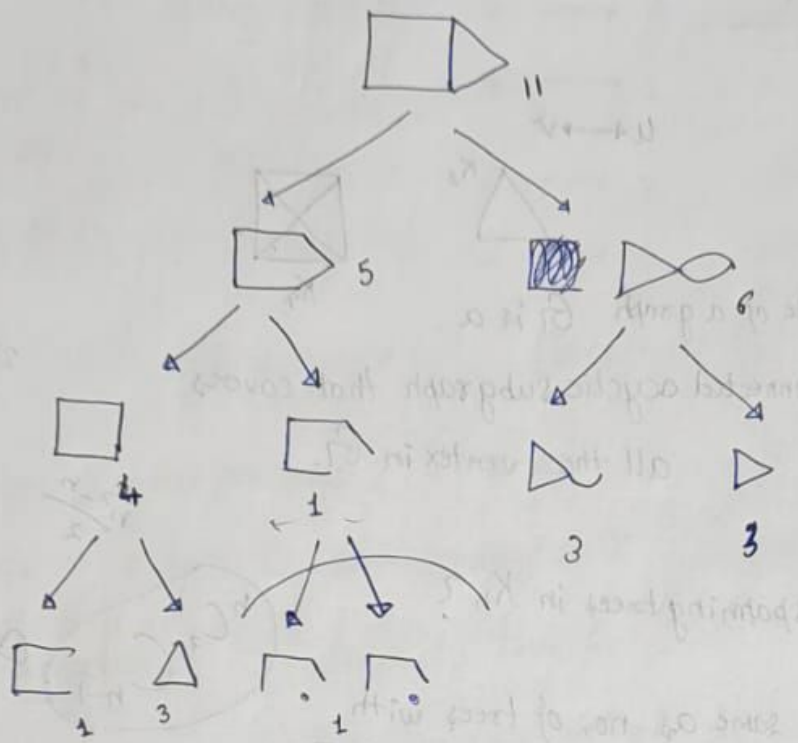
$G' = G \cdot e$

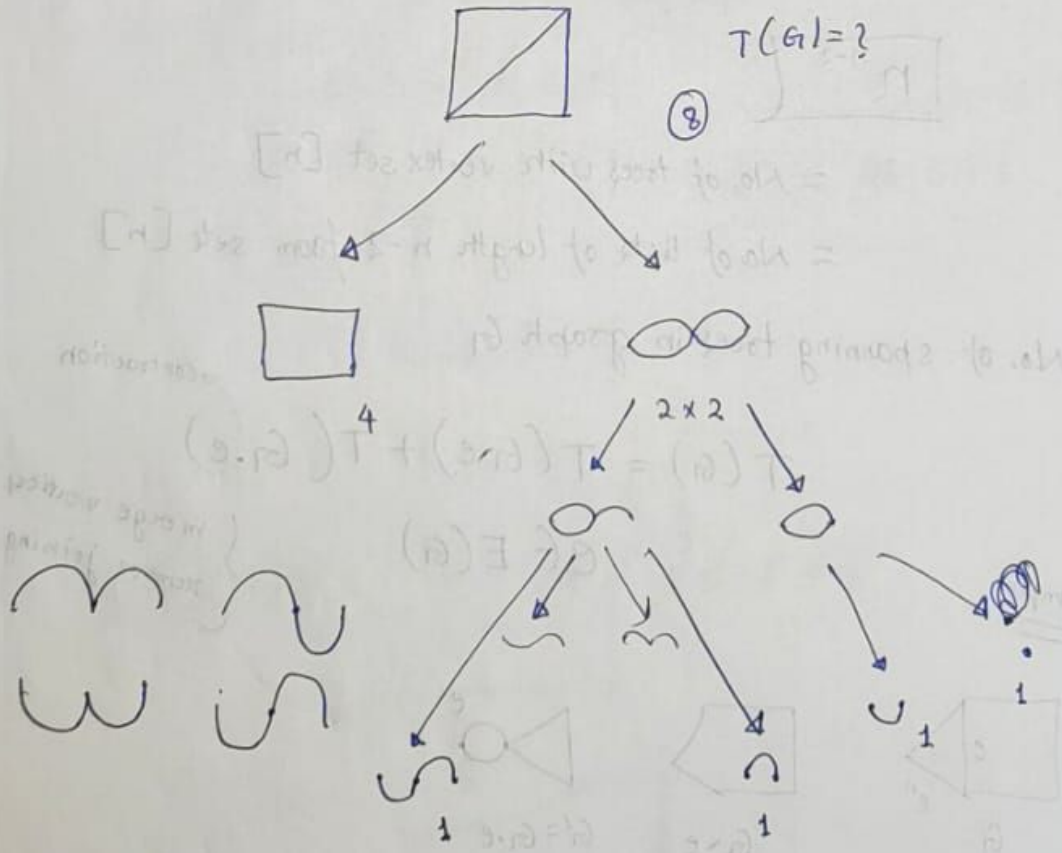
$T(G''-e'') + T(G''.e'')$

$\square = 4 \quad \square = 1$

$T(G'-e') + T(G'.e')$

$= 3 \quad = 3$


11
5
6
4
1
3
3
1
3
1


T(G)=?
8
4
2x2
1
1
1
1

Matrix Tree Theorem

Given a complete Graph G, with vertices $v_1, v_2, \ldots, v_n$.

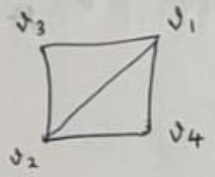
a_{ij} is the number edges with end points v_i and v_j.

A matrix Q is a $n \times n$ matrix where $Q(i,j) = -a_{ij}$ iff $i \neq j$

$Q(i,j) = \deg(v_i)$ iff $i = j$

A reduced matrix Q^* removes s-row and t-column from Q.

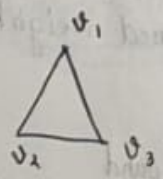
$$\tau(G) = (-1)^{s+t} \det Q^*$$



$$Q = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccc} v_1 & v_2 & v_3 & v_4 \end{array} \\ \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 2 & 0 \\ -1 & -1 & 0 & 2 \end{bmatrix} \end{array} \Rightarrow Q^* = \begin{bmatrix} 3 & -1 & -1 \\ -1 & 2 & 0 \\ -1 & 0 & 2 \end{bmatrix}$$

$$|Q^*| = 3(4) + 1(-2) - 1(+2)$$

$$= 8$$



$$Q = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix} \Rightarrow Q^* = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$$

$$|Q^*| = 4 - (+1) = 3$$

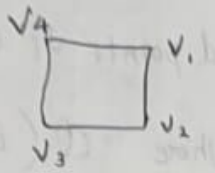
Vertex Coloring

Appl. Scheduling Problems (Exam scheduling, meeting scheduling)

Register Allocation

* K-coloring of graph G,

$$f: V(G) \longrightarrow K$$



$f(v_1) = K_1$

$f(v_2) = K_1$

$f(v_3) = K_2$

$f(v_4) = K_3$

$K = \{v_1, v_2, v_3\}$

* Proper K-coloring iff $v_1 \longleftrightarrow v_2 \Rightarrow f(v_1) \neq f(v_2)$

* Chromatic no. of graph G, $\chi(G)$,

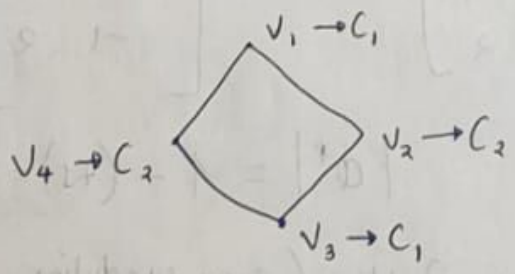
least K s.t. G is having a proper K-coloring

Determining optimal K-coloring of a graph G,

Greedy Approach

- Order the vertices of $G = v_1 \quad v_2 \quad v_3 \quad ---- \quad v_n$
- Assign $\underbrace{\text{a color}}_{\text{least-valued}}$ to vertex v_i which is not assigned to its lower valued neighbour

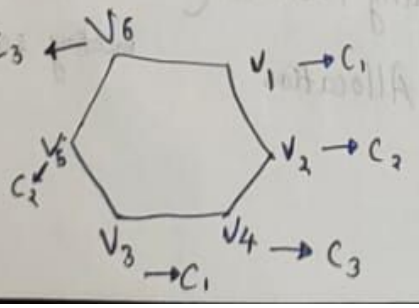
(Is it optimal?)



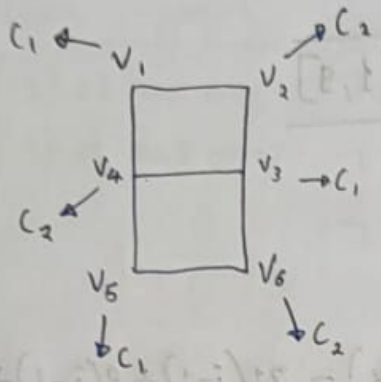
Upper Bound

$$\chi(G) \leq \Delta(G) + 1$$

max vertex degree of G



$\chi(C_6) = 2$



$V_3, V_4, V_1, V_2, V_5, V_6$

Counting no. of proper K-colouring of a graph G.

- all K colours may not used in a K-colouring of a graph
- If we change the color in a K-coloring, it will be counted separately.

$\overline{K}$ $\quad \Delta K_3 \Rightarrow \therefore \overline{K_3}$

$\boxed{\overline{K_n}}$ $\chi(\overline{K_n}; k) = k^n$ $\quad \boxed{K_n}$ ${}^kC_n \; n!$

$\chi(K_n; k) = {}^kP_n$

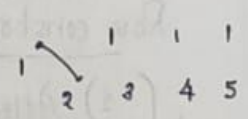
Tree with n vertices

$\chi(T_n; K)$

$= K(K-1)^{n-1}$

1, 2, 3, 4, 5

$K=5, n=2$



If G is a simple graph, $e \in E(G)$ then,

$$\chi(G; k) = \chi(G - e; k) - \chi(G.e; k)$$

③ result = subprocess.run([,], ,)

output = ["s SATISFIABLE", "v 1 -2 3 -4 5 -6 0"]

model = [1, -2, 3, -4, 0, 5, -6, 7, 0, ---]

④ for val in model

if val > 0

$v = val - 1$

$(v \% 1) // 81 + 1$

$(v \% 81) // 9 + 1$

$(v \% 9) // 1 + 1$

$r = (v // 81) + 1$

$c = [(v \% 81) // 9] + 1$

$d = (v \% 9) + 1$

grid[r-1][c-1] = d

$ID(i,j,k) = 81(i-1) + 9(j-1) + k$

$(2,3,5) = 81 \cdot (1) + 9(2) + 5$

$= 103$

CHROMATIC RECURRENCE

$$\chi(G;k) = \chi(G \setminus e; k) - \chi(G.e; k)$$

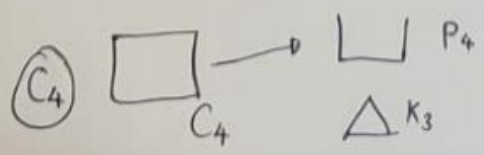
A proper K-coloring in G is a proper K-coloring in $G \setminus e$.

A proper K-coloring in $G \setminus e$ is a proper K-coloring of G if the endpoints u, v of e has disting colors.

=> No. of proper K-colorings in G, can be obtained by subtracting from no. of K-colorings in G.e

[the no. of K-colorings with endpoints of e assigned with some colors] => no. of K-coloring in G.e.

Example: $\chi(G_4; k) = \chi(P_4; k) - \chi(K_3; k)$



$$= K(K-1)^3 - K(K-1)(K-2) = (K-1)(K^3-3K^2+3K-1) - K(K^2-3K+2)$$

$$\cancel{= K^4 - 3K^3 + 3K^2 - K - K^3 + 3K^2 - 3K + 1}$$

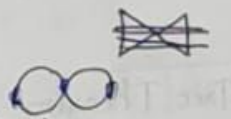
$$= K(K-1)\left((K-1)^2 - (K-2)\right)$$

$$= K(K-1)(K^2-2K+1-K+2) = K(K-1)(K^2-3K+3)$$

G

$$\chi(G;K) = \chi(P_4;K) - \chi(G';K)$$

$$\cancel{= K(K-1)^3 - K(K-1)(K-1)}$$



$$= K(K-1)(K^2-3K+3) - K(K-1)(K-1)$$

$$= K(K-1)(K^2-3K+3-K+1) = K(K-1)(K^2-4K+4)$$

$$= K(K-1)(K-2)^2$$

<u>Laplacian matrix</u>

$$G \to \tau(G) = (-1)^{i+j} \det Q_G^*$$

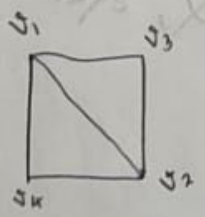
$Q_G^* \to L_G$

$Q_G^* \to$ Laplacian matrix of Graph G

$$Q_G^* = D_G - A_G$$

D_G: Degree Matrix

A_G: Adjacency Matrix



$$D_G = \begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix} \qquad A_G = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Min x,y-cut / Max x,y-internally disjoint paths

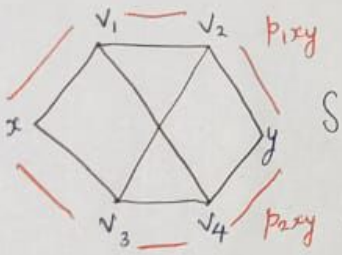
G: $x, y \in V(G)$

Set S of vertices is a x,y-separator / x,y-cut

iff ~~$V(G)$~~ x, y are not reachable, if S is removed from $V(G)$

$$V(G) \setminus S$$

$(G - S)$ does not have any x,y-path



$S = \{v_1, v_3\} / \{v_2, v_4\}$

x,y - internally disjoint paths

Given G, $x, y \in V(G)$

$$p_{1xy} \qquad p_{2xy}$$

p_{1xy} and p_{2xy} are internally disjoint iff

$$\{V(p_{1xy}) - \{x,y\}\} \cup \{V(p_{2xy}) - \{x,y\}\} = \phi$$

Menger's Theorem

$K(x,y) \rightarrow$ min x,y-cut in G,

$\lambda(x,y) \rightarrow$ max no. x,y-internally disjoint paths

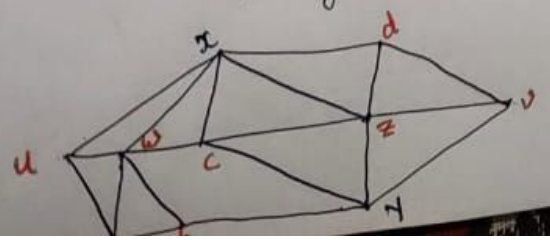
$$K(x,y) \geq \lambda(x,y)$$

$$k(x,y) == \lambda(x,y)$$

$$k(x,y) \leq 4 \longleftrightarrow \lambda(x,y) \geq 4$$

$$k(x,y) \geq \lambda(x,y)$$

$$\Rightarrow K(x,y) = \lambda(x,y)$$



$|M| \leq |e_G|$ $\quad e(G) = m$

$|M| \leq \frac{|n_G|}{2}$ $\quad n(G) = n$

$O(mn)$

→ How many times augumenting path algo needs to be called? $(n/2)$

→ Each time a vertex is explored, how many times it's considered?

* Stable Matching (Gale Shapley Algo)
* Max^m Matching in General Graph (Edmond Blossom Algo)

Preference order

Men	Women
x: a>b>c>d	a: z>x>y>w
y: a>c>b>d	b: y>w>x>z
z: c>d>a>b	c: w>x>y>z
w: c>b>a>d	d: x>y>z>w

- Every man proposes to the highest women on his preference order who has not previously rejected him.
- Each women recieves exactly one proposal (stop & use the matching).
- O.w. every women receiving more than one proposal, rejects all of them except the one that is highest in her preference list.

- Stable Matching (Gale Shapley Algo)
- Maximum matching in General Graph (Edmond Blossom Algo)

Preference Order -

— Gale Shapley Algo

Men		Women	
x :	a > b > c > d	a :	z > x > y > w
y :	a > c > b > d	b :	y > w > x > z
z :	c > d > a > b	c :	w > x > y > z
w :	c > b > a > d	d :	x > y > z > w

- Every man proposes to the highest women on his preference order who has not previously rejected him.
- Each women receives exactly one proposal (stop and use the matching)
- Otherwise every woman receiving more than one proposal, reject all of them except the one that is highest in her preference list
- Every women receiving a proposal says "may be" to most attractive proposal received.

Stable Matching - A perfect matching which doesnot have any unstable unmatched pairs.

It1 -
$x \to a$
$y \to a$
$z \to c$
$w \to c$

$a \to x, y \Rightarrow a \to$ "maybe" x, "reject" y] $(a-x)$ match
$c \to z, w \Rightarrow c \to$ "may be" w, "reject" z] $(c-w)$ match
$\to$ c accepts w (highest in list)

It2 -
$y \to c$
$z \to d$

$c \to$ "rejects" y
$d \to$ "Accept" z] $(d-z)$ match

It3
$y \to b$

$b \to$ "Accept" y] $(b-y)$ match

called ?
$(n/2)$
Considered ?

Algo)

ne order

the matching)
cts all
list.

- Every woman recieving a proposal says "maybe" to most attractive proposal recieved.

A stable matching is a pm which doesn't have any unstable unmatched pairs.

It 1:

$x \to a$
$y \to a$
$z \to c$
$w \to c$

$a \leftarrow x$, $a \leftarrow y$

$a \to$ 'maybe' x
$a \to$ rejects y

$c \leftarrow z$, $c \leftarrow w$

$c \to$ accepts w
$c \to$ rejects z

It 2:

$y \to c$
$z \to d$

$c \to$ rejects y
$d \to$ accepts z

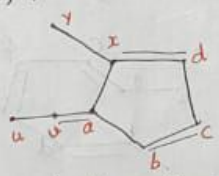
$a - x$ (~~preferable~~)
$c - w$ ✓
$d - z$ (~~preferable~~)
$b - y$

It 3:

$y \to b$

$b \leftarrow y$ accepts y

Blossom, Flower, Stem



G

Matching M

$v - a$
$b - c$
$x - d$

$\boxed{u, d - M\text{-alter. path}}$

Flower: Union of alternating path starting from Given starting vertex u and target vertex v union of u, v - M-alternating paths results a flower.

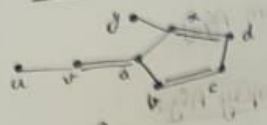
Stem of a flower:

Maximal common path (of even length) is the stem of a flower.

Blossom:

Odd length cycle in the flower is a blossom.

- Edmond Blossom Algo



G

Matching M = xd, bc, va

- **Flower** - Union of alternating path

uvabcd
uvaxd] Two M - alternating paths in G.

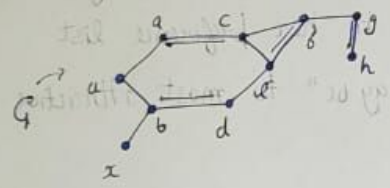
Flower = uv, va, ax, xd, ab, bc, cd

Given a starting vertex 'u' and target vertex 'v'. Union of u-v M alternating paths result a flower. (Paths should be of different parity)

- Stem of a flower - maximal common path (of even length) is the stem of a flower. — uva

- Blossom - Odd length cycle in the flower is a blossom

axdcba



Objective - Find maximum matching

G, M, u

Start exploring from u

Traverse along unmatched edge

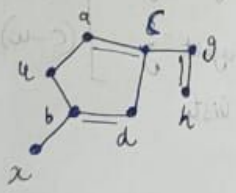
S = {u} ; T = {Φ}

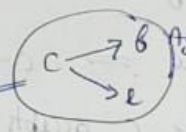
~~S = {u,a,b}~~ , T = {a,b}

S = {u,c,d}

u → a = c
u → b = d

Start from C further





Again saturated & There is a matching edge e & b.

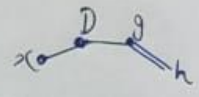
Contract to a single node

Odd length Cycle → Blossom

Now again start from C

S = {u, C, d} T = {a, b}

~~S = {u, C~~ C — checks — d, So already present S - again get blossom.



u = a — c = b — e = d — b = x

Edmond's Blossom Algo

Input: A Graph G, a matching M, a M-unsaturated vertex u

Initialisation: $S = \{u\}$, $T = \phi$

Procedure: ① If there is no unmarked vertex in S, $(S - \phi)$ report no augmenting path

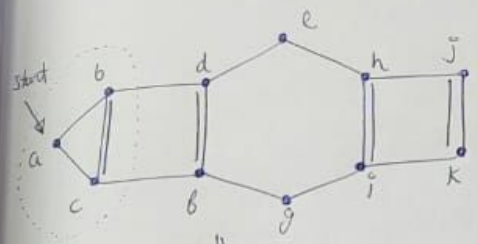
② Otherwise select a $u \in S$, successively explore all $y \in N(u)$

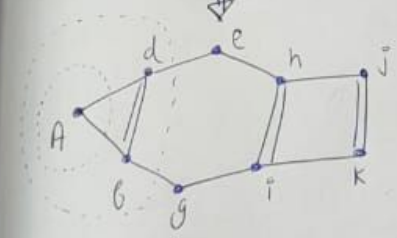
③ If y is unsaturated, repeat a u-y M-augmenting path (expand all blossoms) traceback from y to get M-augmenting path

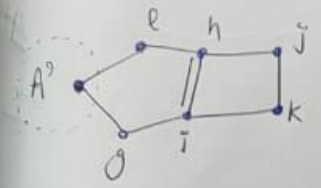
④ If y is saturated, there is a w s.t $yw \in M$, $S = S \cup \{w\}$, $T = T \cup \{y\}$

⑤ If $y \in S$, report blossom, stop exploring y, Contract all vertices in S & T to a single vertex Y, include Y in S, explore from Y.

⑥ After exploring all Y, mark u.







Itr1 – $S = \{a\}$, $T = \phi$

$a \to b$, $a \to c$ ⟶ b is M-saturated

$S = \{a, c\}$ $T = \{b\}$

c is reachable from b (Information recorded)

$c \in S$

Report a blossom

↳ (a – b – c) consistitutes odd length cycle

⇓

Contract into a single vertex A

$S = \{A\}$, $T = \phi$

Now Explore A, $A \to d$ — saturated, $A \to f$

$S = \{A, f\}$, $T = \{d\}$

$f \in S$ ⟶ So again Contract.

$S = \{A'\}$, $T = \phi$

Now Explore A', $A' \to e$ — unsaturated so M-augmented path from e – A'

$A' \to g$

↓

Traceback from e.

Edmond's Blossom Algo.

~~Input: 1. A Graph G, a matching M, a M-unsaturated in S / S = ϕ, stop, repeat no M-augumenting path.~~

Input : A Graph G, a matching M, a M-unsaturated vertex u,

Initialization: $S = \{u\}$, $T = \phi$

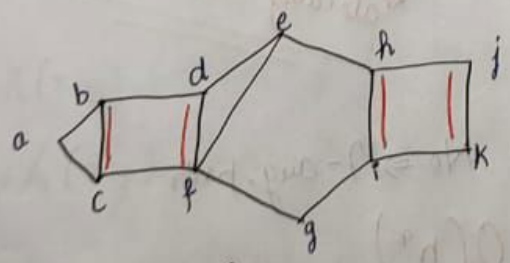
Procedure: 1. If there is no unmarked vertex in S / $S = \phi$, stop, repeat no M-augumenting path.

2. O.w. select a $u \in S$, successively explore all $y \in N(u)$

3. If y is unsaturated, repeat a u-y M-aug. path (possibly expand all blossoms) traceback from y to get M-aug. path.

5. If $y \in S$, report a blossom, stop exploring, Contract all vertices in S & T to a single vertex Y, include Y in S, explore from Y.

6. After exploring all Y, mark u.



$M = \{bc, df, hi, jk\}$

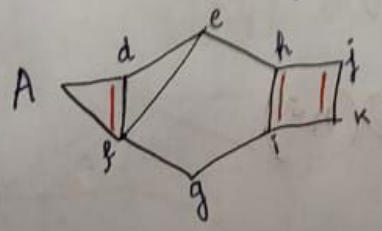
$S = \{a\}$, $T = \phi$

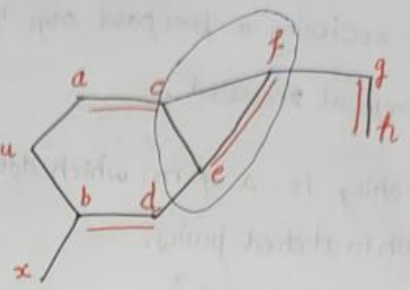
Step 1: $a \to b$ b is M-saturated

$S = \{a, c\}$ $T = \{b\}$

$a \to c$

Step 2: $c \in S$, report a blossom, Compact S & T into a single vertex A, $S = \{A\}$





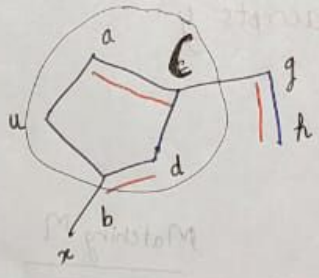
G Objective: Find max^m matching.

input: G, M, u

$M = \{ac, bd, ef, gh\}$

$S = \{u\}$, $T = \{\phi\}$

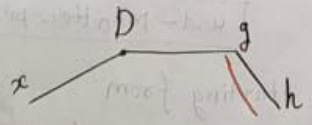
$S = \{u, c, d\}$, $T = \{a, b\}$

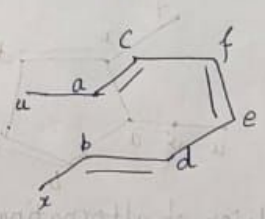


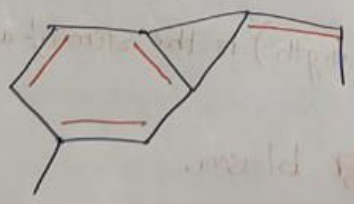
$S = \{u, C, d\}$

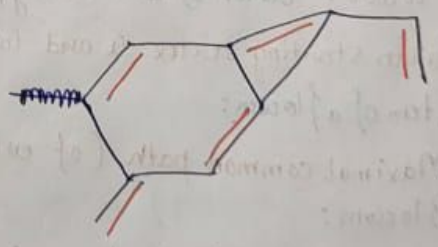
$T = \{$

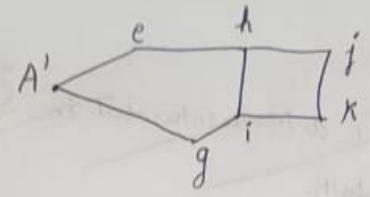
$S = \{u, C, D\}$











Step 3: explore A,

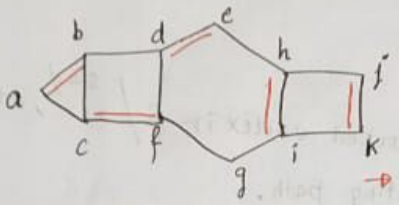
$A \to d$, d is M-saturated

$S = \{A, f\}$ $T = \{d\}$

$A \to f$, $f \in S$, report a blossom,

Contract to a vertex A'

$S = \{A'\}$



Step 4: Explore A',

$A' \to e$, e unsaturated,

there is an $e - A'$ M-aug. path, traceback from e.

$\to eA'$

$\to edfA$

$\to \boxed{edfcba}$

$\Rightarrow S = \{g\}$, $T = \phi$

$g \to i$ $S = \{g, h\}$ $T = \{i\}$

$g \to f$ $S = \{\cancel{g}, h, \cancel{c}\}$ $T = \{i, f\}$

$h \to$ $S = \{\cancel{h}, c\}$

e $S = \{\cancel{e}\}$

already saturated

Complexity

$S = \phi \Rightarrow$ No $\Rightarrow$ M-aug. path

$|V| \times (|V| \times |E|)$ $O(n^4)$

$O(|V| \times |E|) = O(n^3)$

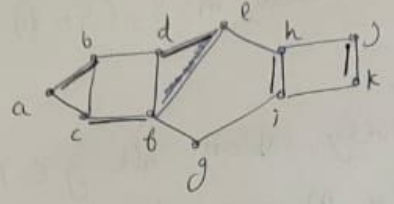
~~eA'~~ → ~~A' → d f A~~ → ~~b c a~~

eA' → A' → f d A → c b a

e d f c b a

~~e d f b c a~~

e f d c b a



e f d b c a

Complexity - $|V| * (|V| * |E|)$, $|V| = n$

→ $O(n^4)$ $\xrightarrow{\text{Optimise}}$ $O(|V| * |E|)$

$= O(n^3)$